

ClientMatrix — College Project Report

Project type: Full-stack Multi Tenant SAAS Platform (college / academic demonstration)

Local demo: Frontend `http://localhost:5173` Backend `http://localhost:5001`

1. Project overview

ClientMatrix is a platform where:

- Client see their overall progress of all Social Media accounts at a single place.
- Super Admin manage approval, KYC Documents verification, Subscriptions and funds related operations.
- Admin(Manager) manage tasks, assign task to Content Manager, Works as a mediator between Client and Content Manager.
- Content Manager builds the user friendly content accordingly to client needs with the use of AI features.
- Support and Raised complaints are treated individually raised by clients.

The system is built end-to-end as a **college project / demo product** (not a production commercial launch). Core marketplace, role-based dashboards, payments (test/demo), reports, and Local Database are implemented and runnable locally.

2. Problem statement

Digital marketing agencies often struggle with fragmented tools, manual tracking, and weak access control when managing multiple clients' social media channels, subscriptions, and creative workflows.

Client Matrix Solution:

- **Unified Multi-Tenant Dashboard:** Centralizes cross-platform social analytics (Instagram, YouTube, X, Facebook, LinkedIn) for agencies and enterprise clients.
- **Strict KYC & Access Control:** Keeps client accounts locked until compliance documents are manually verified by the Super Admin.
- **Automated Global Billing:** Integrates Razorpay subscription tiers with multi-currency support (INR, USD, EUR, GBP, JPY).
- **Creative Workflow Pipeline:** Features a dedicated graphic designer workspace with an AI 4-grid prompt generation and asset submission workflow.

3. Objectives

Objective	Status
Public marketplace (browse, detail, contact, customize)	Done
Multi-role session based authentication	Done
Admin / Super Admin operations	Done
Support and raising tickets	Done
Client Operations	Done
Seeded demo data for viva / presentation	Not Done

4. Tech stack

Layer	Technology
-------	------------

Frontend	Html, Tailwind CSS, JavaScript(JS)
Backend	Python(Django)
Database	Sql Lite
Auth	Django built-in session-based auth system
Payments	Razorpay (test) + demo pay path for college demo
Exports	ExcelJS, PDFKit

5. User roles in the system

Role	What they can do
Super Admin	Full platform control: users, settings (Razorpay/Cloudinary/GST), payment reports, audit log, all ops modules
Agency Staff Admin	Handles administrative operations within the agency.
Client	Operates a KYC-verified custom portal to track multi-channel social performance and billing.
Content Manager	Manages assigned creative deliverables and utilizes the AI prompt generation workspace.

6. Major modules / features (what is in this project)

- 61
- Authentication & KYC Registration Module:** Handles role-based sign-in for users and a secure client registration form requiring compliance document uploads (PAN/GST), which remain locked until Super Admin approval.
- 62
- Super Admin Dashboard:** Provides central control for reviewing and approving pending tenant KYC requests, monitoring active subscriptions, and viewing revenue metrics.
- 63
- Client Portal & Analytics Dashboard:** Enables clients to manage OAuth 2.0 API connections for social channels (Instagram, YouTube, Facebook, and X) and monitor cross-platform performance metrics.
- 64
- Multi-Currency Razorpay Subscription Engine:** Manages tiered pricing plans (Silver, Gold, and Diamond) with dynamic currency switching (INR, USD, EUR, GBP, JPY) and clear feature comparison grids.
- 65
- Graphic Designer Workspace & AI Generator:** Features a 4-variation AI prompt image generator and a task tracking pipeline for submitting creative deliverables to managers.
- 66
- Support Helpdesk:** A dedicated ticketing system where clients can report issues regarding social integrations, billing, or KYC compliance.
-

7. System architecture (simple)

```
Browser (Tailwind HTML Templates / Client Views)
| REST / OAuth 2.0 / Session Auth
▼
Backend Server (Node.js / Express or Django API)
|
▼
MongoDB Atlas (Multi-Tenant Database & KYC Records)
+
Razorpay (Subscriptions) · Social OAuth APIs · AI 4-Grid Prompt Engine
```

`preview/` .

8. Project structure *The production UI for presentation is final product.*

```
ClientMatrix/
├── backend/           # Python/Django, SQL (Models, Views, Serializers)
├── preview/          # Live Tailwind/React UI used for demo
├── docs/             # API docs, walkthrough, checklists, reports
├── CREDENTIALS.md    # Demo logins and role credentials
├── PROJECT_PLAN.md   # Original project plan and roadmap
├── FRONTEND_PLAN.md  # Frontend planning and design notes
├── TRACKER.md        # Development progress tracker
└── README.md         # Project overview and setup guide
```

9. Team contribution & work division

This section is included for college evaluation honesty: **who contributed what**, and what was actually shipped in the final project.

9.1 Team members

Name	Claimed / intended role	Actual contribution to final project
Shubham Gupta	Lead + Frontend	33.3% (Frontend Dashboard & Drafting)
Yash Srivastava	Frontend & Support Backend	33.3% (Frontend Dashboard of Admin-Manager & Support backend)
Ayush Kumar	Backend	33.3% Backend (But not working properly but considered in project)

92 Summary table (for evaluators)

Area	Who delivered the final working version
Project idea / planning docs	Shubham Gupta (primary)
Backend API + database	Ayush Kumar
Role dashboards	Shubham Gupta
Payments / invoices / wallet	Shubham Gupta
Admin(Manager) Dashboard	Yash Srivastava
Demo seed	Ayush Kumar
Documentation	Shubham Gupta

10. What was intentionally kept simple (college scope)

- Treated as **college / demo project** (not full commercial launch)
- Live Razorpay production, full KYC, legal pages, and production deploy are **out of scope** for this submission (see `docs/LAUNCH_CHECKLIST.md` if asked)

11. Learning outcomes

- Full-stack product thinking (roles, marketplace, money flow)
- REST API design with Python and role middleware
- Sql Lite modelling for Database
- Operational UI
- Honest team contribution documentation for academic evaluation

12. Conclusion

Client Matrix eliminates operational fragmentation by unifying multi-channel social management, KYC security, and AI creative workflows into a single platform. Backed by Django, it is fully architected for enterprise-grade scale and automated global execution.

Document prepared for college viva / project file • ClientMatrix • 2026

